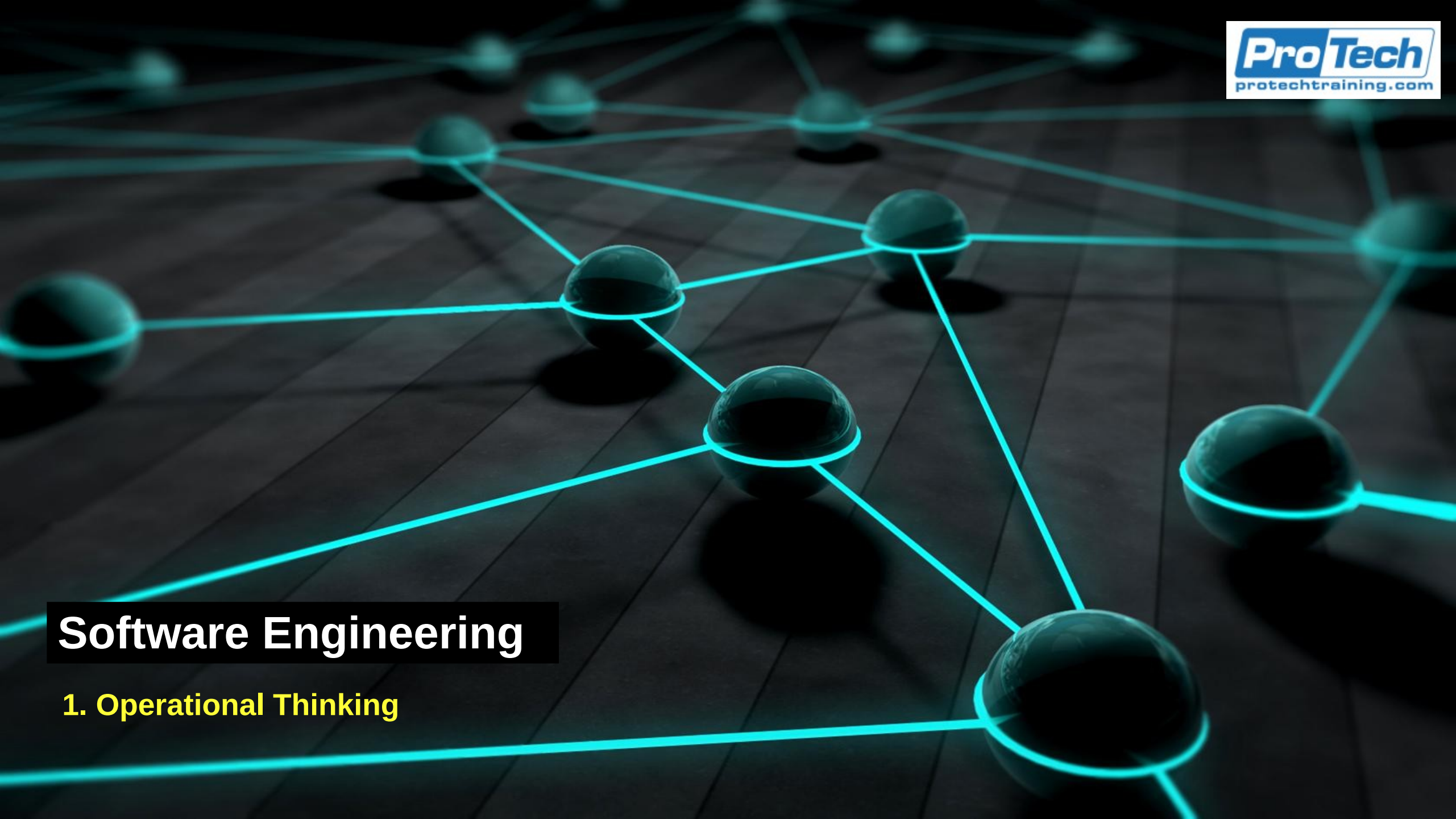


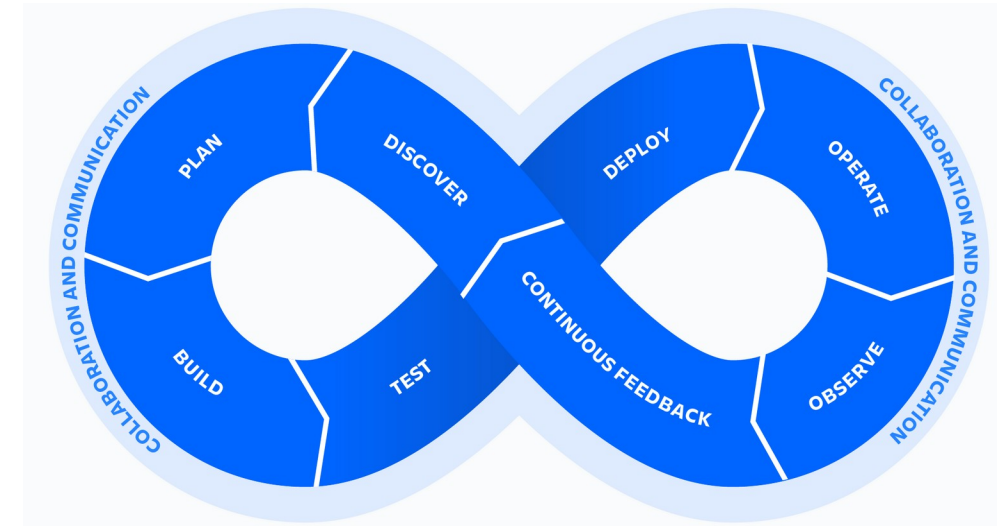


Software Engineering

1. Operational Thinking

DevOps

- DevOps is the discipline that links building software to running it reliably in production
 - Ties the development skills from previous weeks to the real world operations of software
- Operations and development are each half of one continuous engineering effort, not a separate world.
- DevOps provides the a model of
 - How development and operations differ
 - How they fit together.



Origins - The Wall of Confusion

- Historically, development and operations were separate teams with separate goals and little shared context.
 - Developers were rewarded for shipping change quickly, while operators were rewarded for keeping systems stable.
- The handoff point between them became known as the wall of confusion, where finished code was thrown over to whoever had to run it.
 - The conflict occurred because each side was optimizing for the incentives it had been given
- The Dev and Ops worlds were radically different
 - Dev was focused on software and writing code
 - Ops was focused on the physical infrastructure, installing software on physical devices, running network infrastructure, etc
 - Each had their own unique tool-sets, methodologies and processes

Origins - The Wall of Confusion

- Organizations literally separated the two functions into different teams
 - The teams often did not talk until something broke.
- Developers handed a release to an operations team,
- The handoff was the moment things tended to go wrong
 - Which is why people started calling it the wall of confusion
- This was caused by the way the work was structured and rewarded
 - The Dev team was motivated to move fast
 - The Ops team was motivated to never let anything break
 - This creates a continuous point of conflict

The Core Tension - Velocity Versus Stability

- Development naturally pushes for velocity
 - Its purpose is to deliver new features and change.
- Operations naturally pushes for stability
 - its purpose is to keep services available and reliable.
- Every change carries both potential value and potential risk
 - which is exactly why these two goals conflict with each other.
- Both priorities are legitimate
 - A healthy organization keeps them in balance instead of letting one side win outright.

Dev Versus Ops - What Drives the Work

- Primary incentive
 - Development is measured on delivering features and change
 - Operations is measured on preserving reliability and availability.
- Unit of work
 - A developer tends to think in features, commits, and pull requests
 - Operations thinks in services, incidents, and the system over time.
- Time horizon
 - Development centers on the sprint and the release
 - Operations is continuous and spans the full lifecycle around the clock.

The Evolution in Brief

- DevOps
 - Began as the movement to unite developers and operators around shared responsibility for software.
- Site Reliability Engineering
 - Grew out of it as a more formal, engineering-driven implementation of the same goals.
- Platform engineering
 - More recent step, building internal platforms so product teams can operate easily by default.

Dev Versus Ops - Success and Failure

- Success metrics
 - Development tracks velocity, throughput, and lead time
 - Operations tracks availability, recovery time, and adherence to service objectives.
- Stance toward failure
 - In development failure is mostly something caught in tests
 - In operations failure is experienced live with real users, data, and money at stake.
- Cognitive posture
 - Development is constructive and focused on building
 - Operations is anticipatory, defensive, and healthily skeptical.

One System, Two Relationships

- The core difference
 - Development relates to the system as something being built, operations as something being run.
 - Development optimizes for change, while operations optimizes for the consequences of change.
- Neither relationship to the system is complete on its own but complements the other
- As Ops became more reliant and virtualization
 - The tools Ops used started to resemble the tools in Dev, eg. virtual deployments
- DevOps is a set of ideas and practices for integrating the two sides
 - Based on the flow of information between the two sides
 - Created a feedback loop to make each side more effective

Where These Functions Live Today

- In many organizations the work is still divided between product development teams and a traditional operations or system administration team.
- Site Reliability Engineering, often shortened to SRE
 - Applies software engineering practices directly to operations and reliability problems.
- Platform engineering
 - Builds internal tooling and platforms so that product teams can run their own services more easily.
- Engineers take direct responsibility for responding to production issues as they happen.

Where These Functions Live Today

- In many organizations the work is still divided between product development teams and a traditional operations or system administration team.
 - But there are several specific new engineering disciplines designed to bridge the gap
- Site Reliability Engineering (SRE)
 - Applies software engineering practices directly to operations and reliability problems.
 - Leverages the fact that a lot of modern operational environments are primarily virtual
- Platform engineering
 - Builds internal tooling and platforms so that product teams can run their own services more easily.
 - The shared tooling allows individual teams to operate their own services without reinventing everything.
 - Ops uses the same tools as Dev and Dev is now getting feedback from the Ops environment

DevOps Blurs the Boundaries

- DevOps deliberately reduces the hard handoff between the people who build software and the people who run it.
- Shared ownership means the same teams increasingly design, ship, and operate their own services.
- The roles on the previous slide are better seen as a spectrum of where operational responsibility sits, rather than rigid professional silos

Resolving the Tension

- DevOps does mediate between velocity and stability
 - Addresses both through shared accountability between Dev and Ops
- Shared goals mean development and operations are measured against the same outcomes rather than competing ones.
- Shared ownership means the team that builds a service is also responsible for keeping it healthy in production.
- Shared on-call means developers feel the operational consequences of their own code, which aligns incentives directly.

You Build It, You Run It

- “You build it, you run it”
 - Captures the idea that the team who writes a service also operates it.
 - When builders respond to operational issues, the quality of what they ship and the ease of running it both tend to improve quickly.
- Shift-left
 - Means moving operational concerns earlier into development, rather than discovering them only in production.
 - Thinking about reliability, observability, and failure during design is cheaper and more effective than bolting them on afterward.
- Essentially applications are designed for operational resilience from the very start
 - Similar to the idea of building security in from the very start

You Build It, You Run It

- The original wall of confusion existed because two teams were given conflicting goals and then asked to hand work across a boundary
 - DevOps removes the boundary rather than refereeing the conflict across it.
- Shows up in three layers
 - Shared goals.
 - Both groups are measured on the same outcomes, such as a service being both fast to evolve and reliable, the incentive to fight disappears.
 - Shared ownership.
 - The team that built a service is the team that runs it, so there is no one to hand the problem to.
 - Shared on-call
 - When the developer who wrote the code is the one who gets paged at two in the morning, the feedback is immediate and personal.
 - Closes the loop between a design decision and its consequences, which is the fastest way to change behavior and implement improvements

- CI/CD is the automated pipeline that carries a change from a developer's commit all the way to production.
 - CI/CD was covered earlier as a development practice
 - But it also has a tight integration to the operational environment
- Operationally, the pipeline is the controlled path that every change must travel
 - This makes it the natural place to put guardrails.
- Automated tests, checks, and deployment gates in the pipeline are how teams let change move fast while keeping risk contained.

- The pipeline as the single controlled road that every change has to drive down to reach production
 - Every change must pass through one path
 - This means then that path is the perfect place to install guardrails
 - This is where operational thinking and the pipeline meet
 - Automated tests catch regressions
 - Security and policy checks catch risky changes
 - Deployment gates and progressive rollout strategies limit how much damage a bad change can do before it is caught
 - Lets a team have velocity and stability at the same time
 - The pile line allows the team to make changes fast and safely
 - Development is vetted against the constraints and requirements of the operational environment

Feedback Loops

- DevOps enables feedback loops from production into development
- Production is not the end of the line
 - DevOps turns it into a source of information that should flow back into how software is built.
 - Incidents, telemetry, and postmortems all carry lessons from the running system back to the people who design it.
- When that feedback genuinely changes future design, the whole system becomes more reliable over time.
- This is itself a systems-thinking idea, a feedback loop, which is exactly what the next section explores in depth.

Feedback Loops

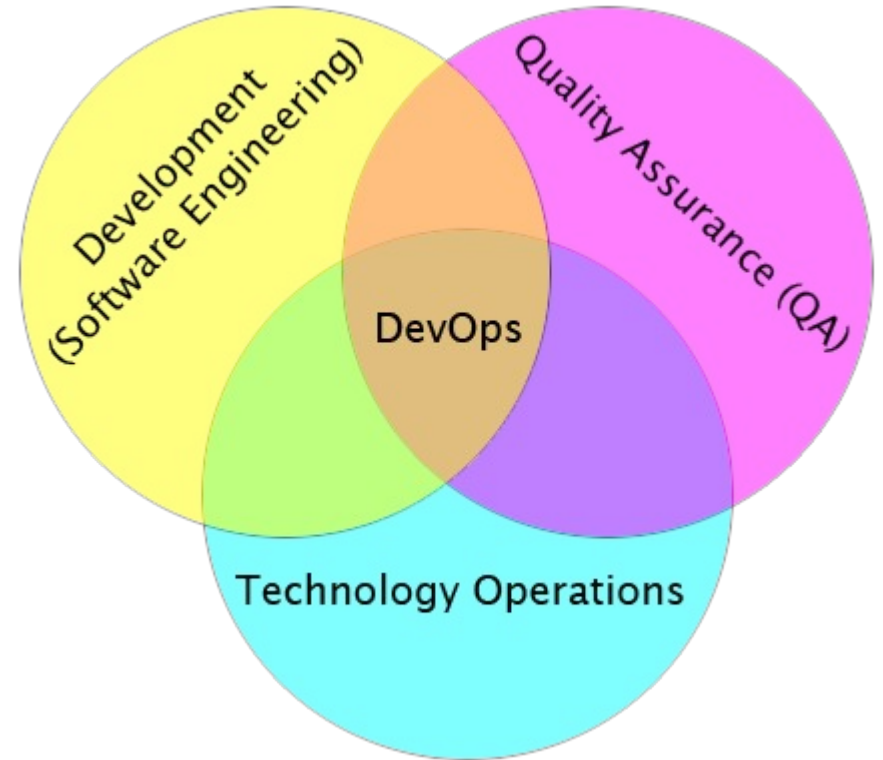
- A running system constantly tells you things through
 - Its telemetry
 - Incidents it produces
 - And through the postmortems written after those incidents.
- With DevOps that information does not just get filed away.
 - It loops back and changes how the next version of the system is designed and built.
 - For example; a recurring failure mode should reshape an architecture decision.
 - Or, a confusing incident should lead to better instrumentation.
 - That returning flow of information is what makes a system get steadily more reliable instead of repeating the same mistakes.
 - This is basic systems control processes as practiced in engineering now applied to software

Operability as a Design Property

- Operability means a system is built from the start to be observed
 - It is destined to fail gracefully, and to be recovered with minimal losses of every type
- These qualities are not added in the operational environment
 - They are design decisions a developer makes while writing the code.
- This is the most direct payoff of operational reasoning for a developer
 - This sort of planning changes how you build software
- It also changes the definition of done for development
 - Instead of just delivering the software as a final act
 - Done now aligns more closely with operations
 - Dev is responding to operational issues to refactor and redesign the software
 - For both sides, done now means the software is running reliably in the production environment

Culture Frames - the Three Ways and CALMS

- DevOps is a culture as well as a set of tools
- A few simple frames capture that culture well.
 - The Three Ways describe flow from development toward operations, fast feedback in the other direction, and continual learning.
 - CALMS is a checklist of culture, automation, lean practices, measurement, and sharing that teams use to gauge maturity.
- These frames emphasize that tools alone do not create DevOps
 - How people work together matters just as much.



Thee Ways

3 Ways of DevOps

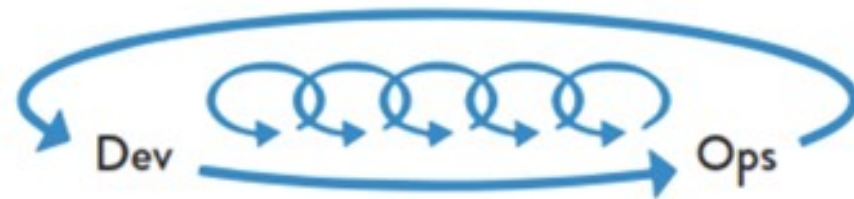
1. Understand and increase flow



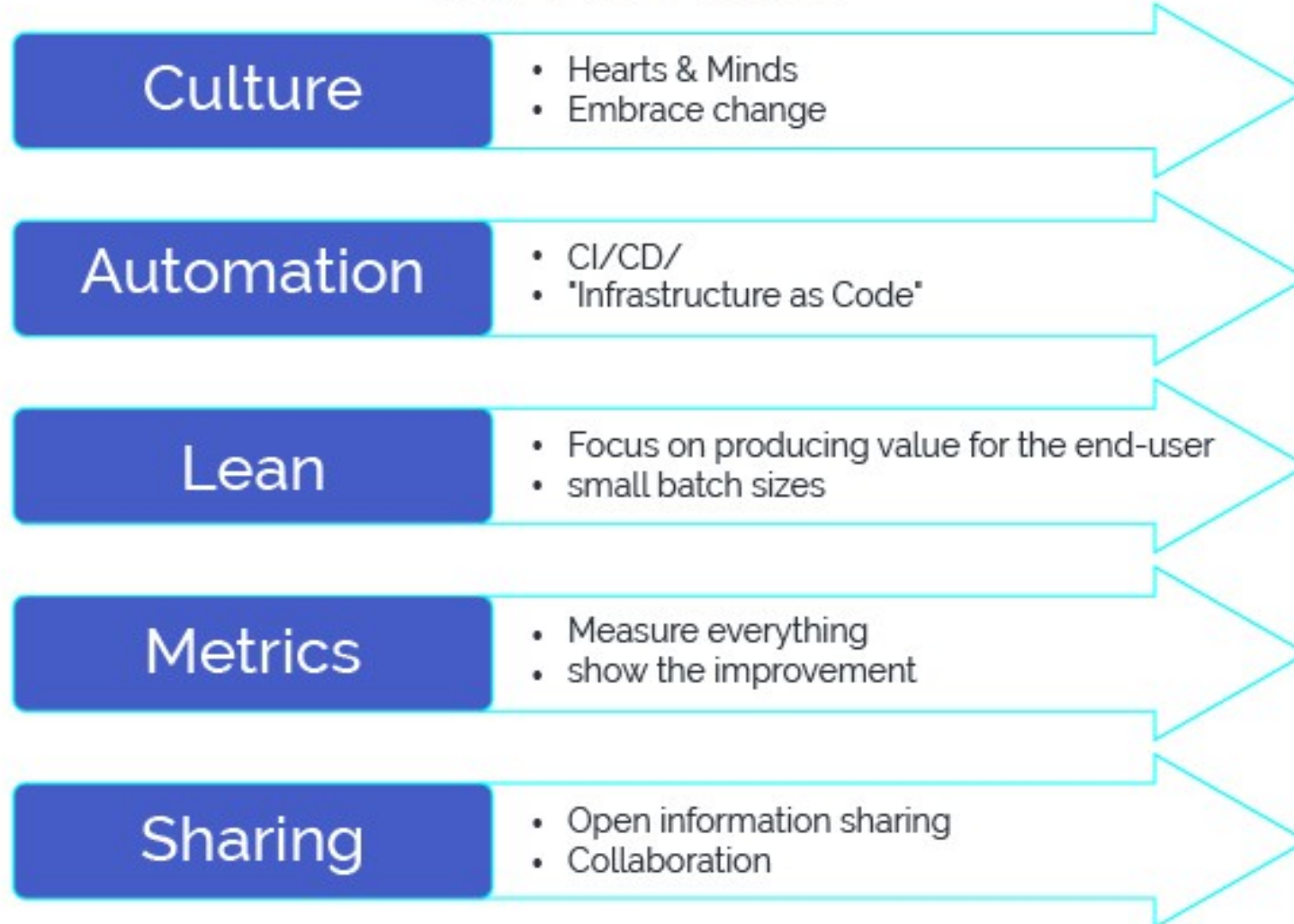
2. Short and constant feedback loops



3. Continual learning and experimentation



CALMS Model



System as Written vs. System as Run

- The system as written is a static artifact
 - Source code that is correct or incorrect before it is executed
 - We can evaluate the correctness and quality of the code without executing it
- The system when running is a living thing
 - That same code executing continuously in a real environment over time.
- Running software does not sit still
 - It is constantly under load, changing, and interacting with the world around it.
- Operational reasoning is fundamentally about the system as run, not the system as written.

The Environment

- A running system is embedded in a network that can be slow, congested, or unreliable at any moment.
- It depends on hardware that can fail, fill up, overheat, or simply run out of capacity.
- It relies on other dependencies and services that can be down, slow, or behaving unexpectedly.
- It serves real users and changes over time, so its behavior is shaped by demand, growth, and age, not just its code.

The Environment

- Environment makes the system as run so different from the system as written
 - None of these surrounding realities show up in the source code
 - The environment is a prime determiner of how the system actually behaves in production.
 - The network is not instant or reliable
 - But in a distributed system, the network is everywhere, so latency and partial connectivity become first-order concerns.
 - The hardware underneath is physical and finite, so it can reach capacity, break down or fail under load or stress.
 - The system works in isolation, because it depends on other services and components
 - Any of which can be slow or unavailable in ways your code did not anticipate.
 - Real users do things you never imagined, at a frequency you did not plan for
 - And the system ages and grows, so its behavior drifts as the small errors and failures accumulate

Why This Reset Is Genuinely Hard

- The instincts that make someone a good developer can be misleading them when used to reason operationally.
- Trusting that correct code will simply behave correctly is reasonable in development and dangerous in production.
- Focusing tightly on one component is a strength when building and a blind spot when operating a whole system.
- The cognitive reset is hard because it is not about learning new facts, but about unlearning trusted habits in a new and different context.

The Shift, Part 1

- The target becomes reliability
- Development asks whether the code is correct, while operations asks whether the service stays reliable, available, and resilient over time.
- Being right once in a test is not the same as staying right continuously in production, which is the harder operational standard.
- Development can often treat behavior as deterministic, where the same input reliably produces the same result.
- But, operations lives in a probabilistic world, where failures are partial, intermittent, and correlated, and “usually” working is the normal state.

The Shift, Part 2

- Reasoning goes global and failure comes first
- Development often reasons locally about one function at a time, while operations reasons globally about how many components interact.
- Most failures originate in the interactions between parts rather than inside any single part, so the global view is not optional.
- Development tends to focus on the happy path, while operations treats failure as something to expect and design for from the start.
- The bar moves from "it works on my machine" to it works in production, at scale, and under adverse conditions.

The Shift, Part 2

- Developers are trained to reason locally
 - To get one function or one module right is exactly the skill they need,
 - Operationally, dangerous behavior almost always emerges from the interaction of many components, not from any single one of them
 - The operational view has to pull back and take in the whole system at once.
- The shift is from the happy path to failure as a first-class concern
 - In development, most of your attention on the case where everything goes right with error handling as a secondary phase.
 - This is necessary to ensure correctness in the logic implemented in the code
 - Operationally, we treat failure as an expected and frequent event that must be planned for deliberately.
 - The operational standard is not that it works in a clean, isolated, single-user environment
 - It must work in production, under real load, with real concurrency, and under adverse and even hostile conditions.

The Shift, Part 3

- The work becomes operate, observe, recover
- The job expands from building software to operating it, observing what it does, and recovering it when it fails.
- Success is no longer just shipping a working build, but keeping the running system healthy across its whole life.
- The response to a problem shifts from a one-off individual fix to improving the systems and processes around it.
- Operations asks not only how to fix this instance, but how to stop the entire class of the underlying problem from recurring.

The Shift, Part 3

- Building is only the opening phase.
 - The ongoing work is
 - Running the system
 - Having visibility into what it is doing, and being able to bring it back to health when something goes wrong.
- When a developer hits a bug, the instinct is to fix that bug and move on.
- The operational instinct is broader and more durable.
 - It asks what about the system or the process allowed this to happen
 - And how can we prevent the whole category of problems from occurring rather than just this one instance.
 - For example, corrupted data from a service the application calls.
 - If the conditions occurred once, can they occur again, and how do we identify when this happens?
 - The operational goal is to apply a fix at the system rather than the symptom, which is how operational maturity compounds over time.

Production Is a Different Country

- Production runs at a scale and concurrency that local development simply can never reproduces
- In production, real money and real customer data are on the line, so mistakes carry genuine consequences.
- Failures in production are usually partial rather than total, and adversaries are actively probing for weaknesses.
- Production problems arrive at the worst times, when judgment is hardest and analytic capabilities are not at their peak

Production Is a Different Country

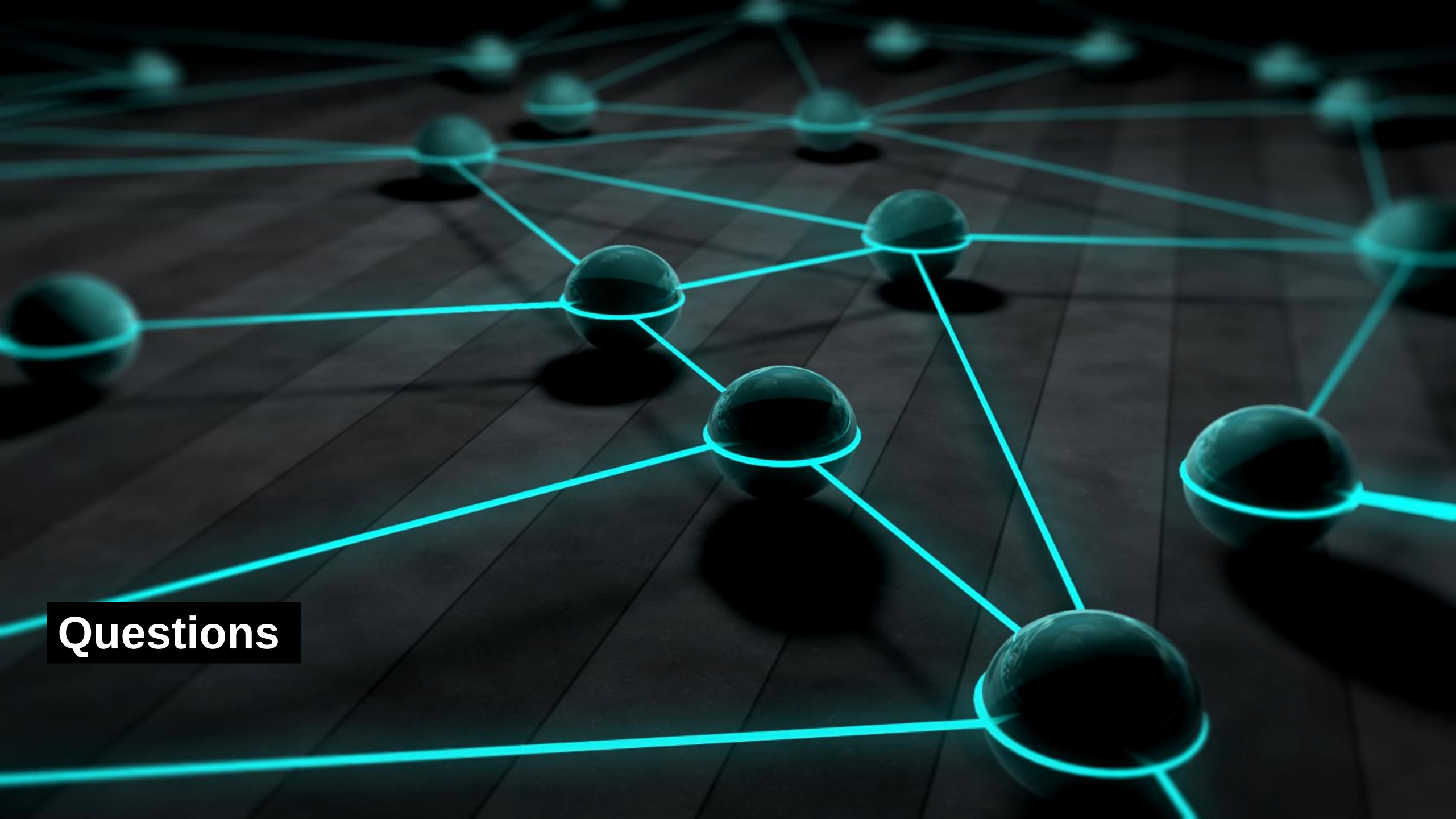
- Applications that work perfectly with one user on a laptop behave entirely differently when thousands of users hit them at once
 - Many bugs only ever appear under that concurrency.
- In production there is real money moving and real customer data at risk,
 - An error is not an inconvenience but a potential financial loss or loss of trust event.
- Failures tend to be partial and confusing rather than clean and total
 - Unlike a test environment, production has people actively trying to break or exploit it.
- Production problems do not respect business hours.
 - They often surface at three in the morning, during a traffic spike, when the on-call engineer is tired and the pressure is highest.
 - This is why we design for failure and recovery ahead of time rather than improvising under stress.

How Complex Systems Fail

- Richard Cook's short essay *How Complex Systems Fail* captures the operational worldview in a few striking claims. (In the resources folder)
- It argues that complex systems are always running in a partially broken, degraded state, even when they appear healthy.
- It argues that failure is the normal background condition, held at bay by the constant work of the people who run the system.
- It argues that safety is not a fixed property of any one part, but something that emerges from the whole system in action.

How Complex Systems Fail

- At any given moment, multiple small flaws and faults are present
- The system keeps working, more or less, in spite of these latent problems
- Failure is the normal state of affairs, continuously held at bay by the active effort of the people and mechanisms operating the system
- The system is not naturally stable and can be occasionally broken.
- In operations, a system is natural tends toward failure and continuously has to be rescued.
 - “Entropy always increases in a system that is left alone.”
- Safety is emergent rather than located in any single component
 - You cannot point to the safe part of the system, because safety is a property of the whole system operating together.



Questions